

Multiprocessadores

João Canas Ferreira / António José Araújo
Abril 2025

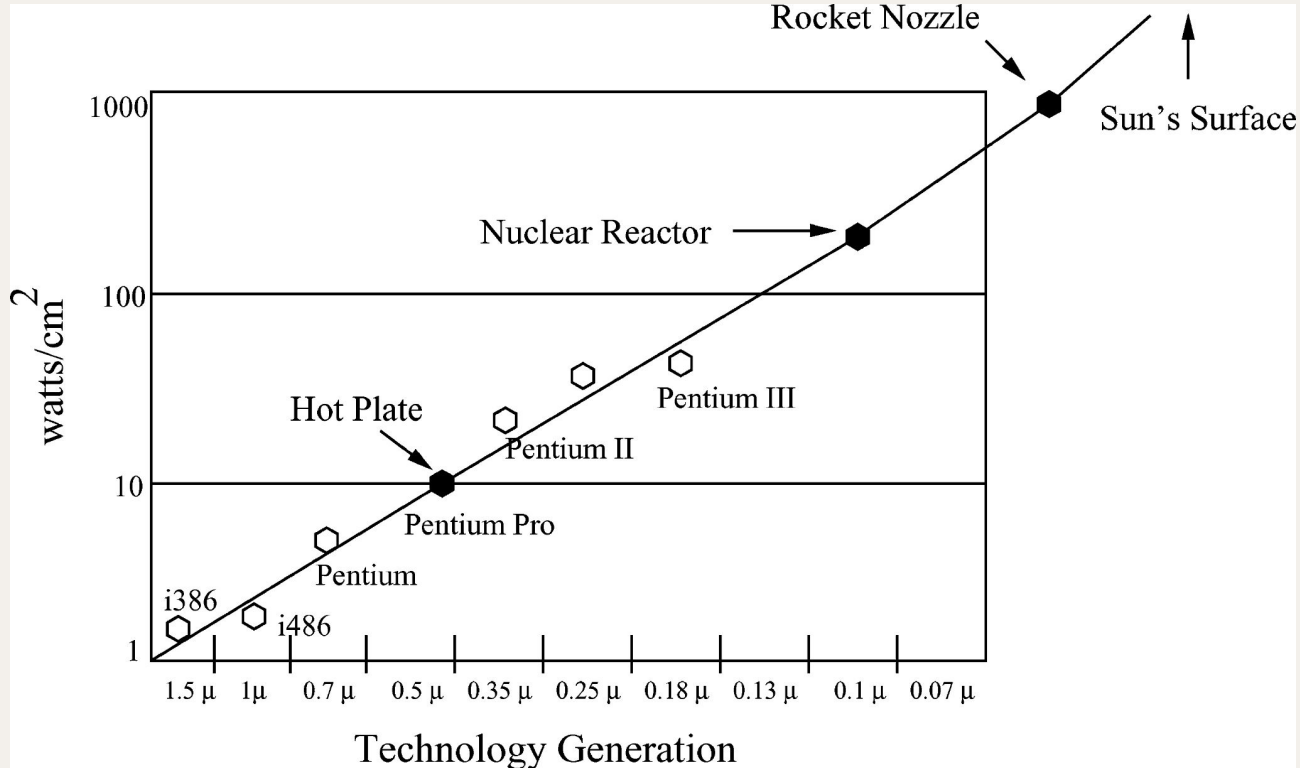
Dissipação de energia

Dissipação de energia em circuitos integrados

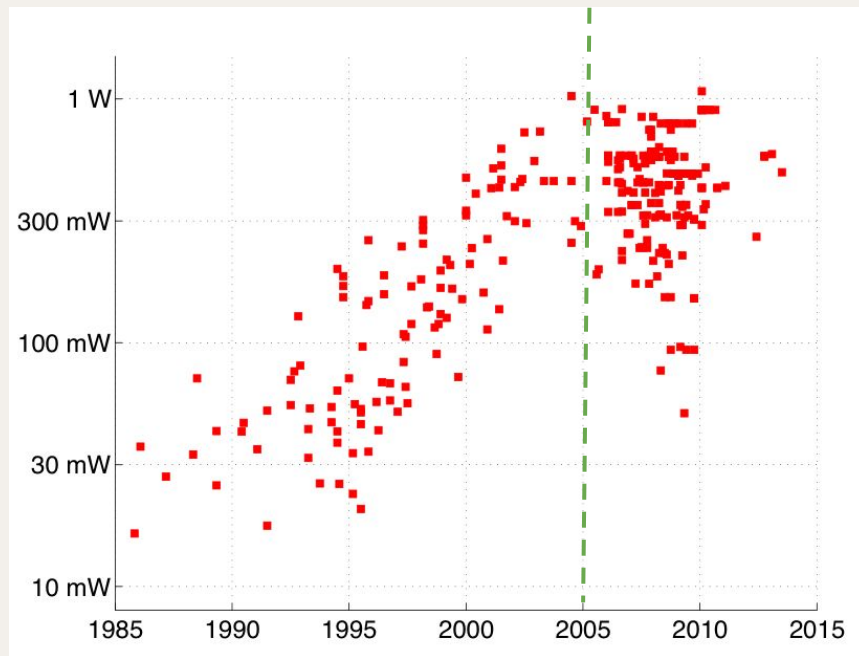
- Unidade de potência: W (watt) Unidade de energia = J (joule)
- Para um intervalo de tempo Δt , a relação entre energia e potência é: **$E = P \times \Delta t$**
- Uma operação lógica básica (AND de 2 entradas) dissipa em tecnologias CMOS atuais cerca de 1×10^{-15} J (1 fJ)
- A potência dissipada por uma porta lógica elementar depende da frequência F:
 - $P_{\text{gate}} = 1 \text{ fJ} \times F$
 - Para um CPU a 1 GHz e com 10^8 (100 milhões de) portas lógicas: $P_{\text{CPU}} = 100 \text{ W}$
- A densidade de potência (potência por unidade de área) é frequentemente mais relevante que a potência total: pontos quentes localizados.
 - Densidade de potência de CPUs desktop de alta performance: 0,5-1,5 W/mm²
 - O número de transístores continua a aumentar segundo a Lei de Moore, mas a densidade de potência tem sido limitada a valores entre 0,5-1,5 W/mm² desde meados dos anos 2000 $\Rightarrow$ limitação da frequência.



Evolução da densidade de potência (1999)



Evolução histórica da densidade de potência



Horowitz, Mark. 2014. «Computing's energy problem (and what we can do about it)». Em 2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC), 10–14.

<https://doi.org/10.1109/ISSCC.2014.6757323>.

Aumento de temperatura

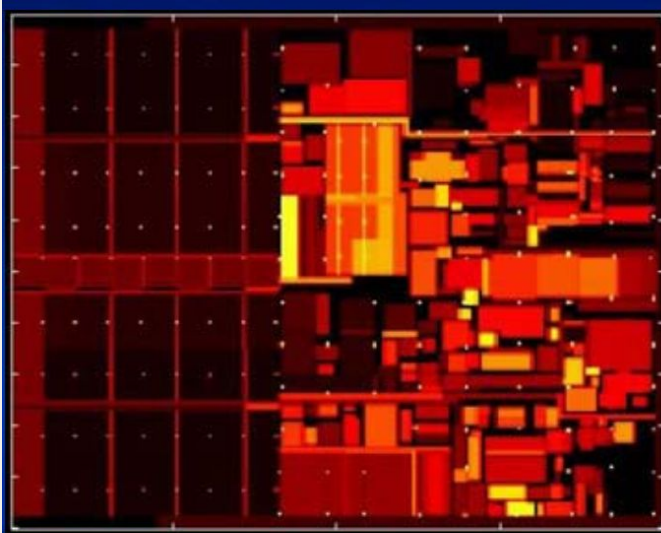
- A potência dissipada (na forma de calor) leva a um aumento da temperatura no interior dos circuitos integrados.
- O aumento de temperatura ΔT causado pela potência dissipada é:

$$\Delta T = P \times R_{\theta JA}$$

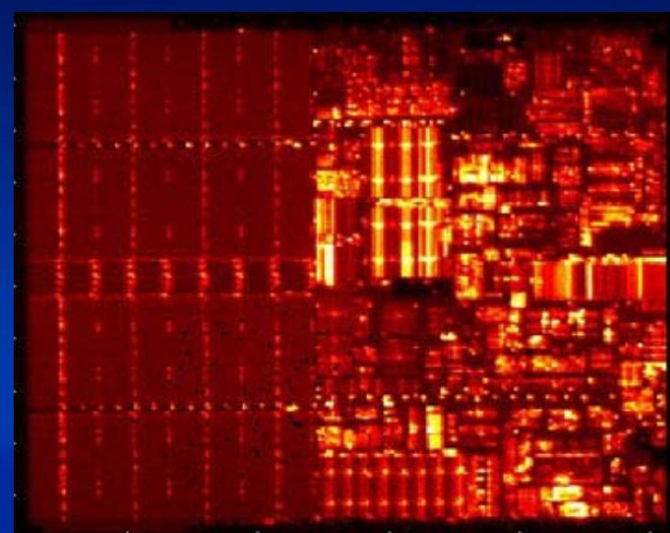
em que $R_{\theta JA}$ é a resistência térmica do transistor para o ambiente (em °C/W)

- CPUs de *desktop* de alta potência: $R_{\theta JA}$ entre 0,2 e 0,5 °C/W
- CPUs de portátil: 0,5-2,0 °C/W (com ventilação forçada)
- Fatores que afetam a $R_{\theta JA}$ (entre outros):
 - Tamanho do circuito e do encapsulamento
 - Presença ou ausência de dissipadores de calor
 - Fluxo de ar no ambiente
 - Utilização de TIM (*Thermal Interface Materials*)

Exemplo de um mapa de temperatura



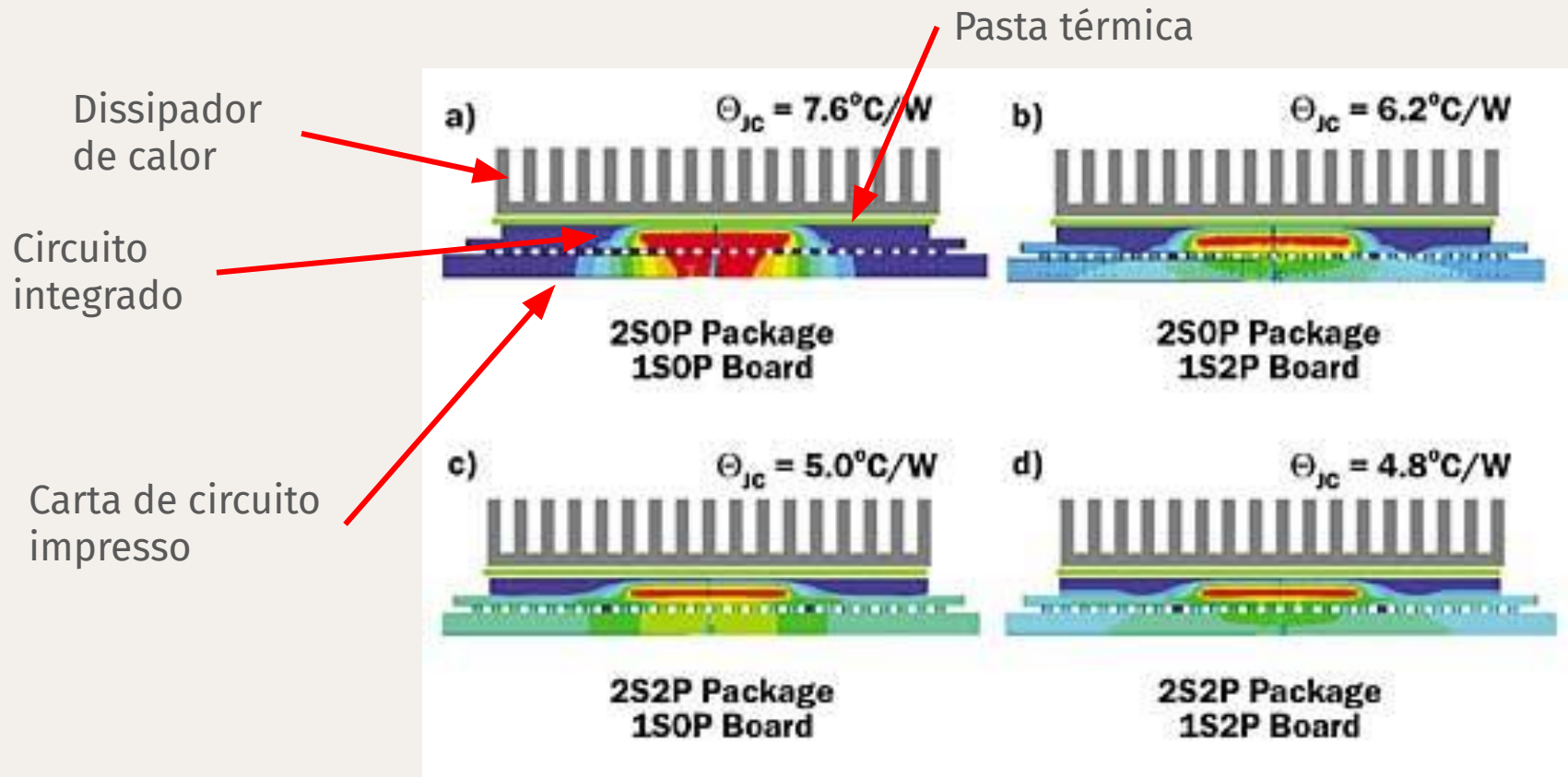
Simulated power density



**Infrared Emission
Microscope measurement**

D. Genossar and N. Shamir "Intel® Pentium® M Processor Power Estimation, Budgeting, Optimization and Validation", Intel Technology Journal 5/2003

Exemplos de diferentes comportamentos térmicos



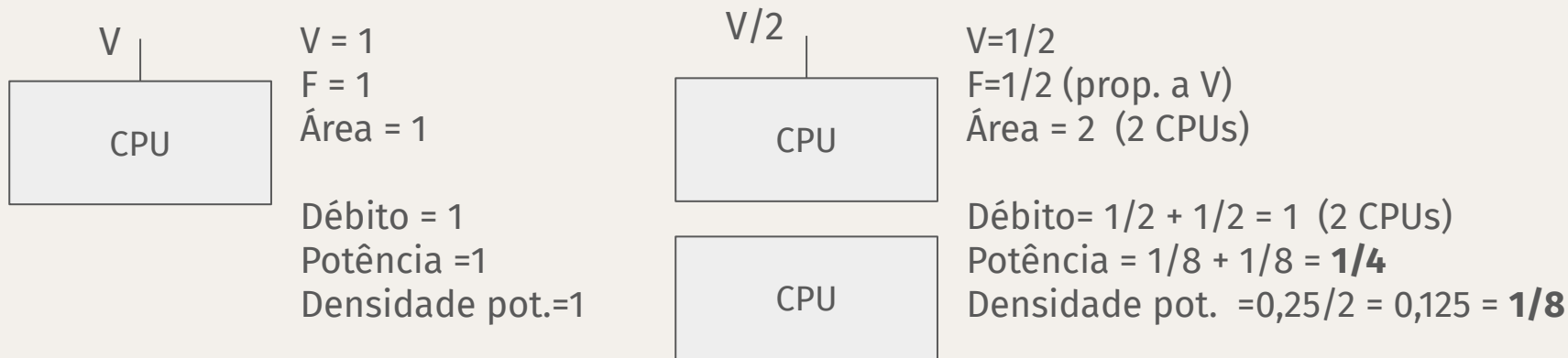
Impacto da tensão de alimentação dos circuitos

Para a mesma tecnologia

- A frequência é proporcional à tensão de alimentação (na gama [1V; 3V])
 - Reduzir tensão de alimentação para metade → reduzir frequência máxima para metade
- Assumindo processadores otimizados para frequência, a potência dissipada é (aproximadamente) proporcional ao cubo da tensão de alimentação.
 - Processador 1 dissipa potência P_1 e atinge frequência F_1
 - Processador 2 dissipa potência P_2 e atinge frequência F_2

$$\frac{F_1}{F_2} \approx \sqrt[3]{\frac{P_1}{P_2}}$$

Usar paralelismo para reduzir a potência dissipada



Se uma única tarefa, executada num CPU com tensão de alimentação V e frequência F , puder ser dividida em duas tarefas independentes, cada uma operando a metade da tensão e frequência:

⇒ o desempenho mantém-se o mesmo, mas a densidade de potência reduz-se por um fator de 8 e a potência de um factor de 4.

Arquiteturas multi-core

Vantagens

- Melhor eficiência energética.
- Ultrapassar a "ILP wall"
 - Refere-se ao limite prático na quantidade de paralelismo que pode ser extraído de um único fluxo de instruções.
- Maior desempenho em aplicações paralelas.
- Melhor resposta do sistema.
- Melhor escalabilidade.

Desvantagens

- Complexidade de programação.
- “Overhead” de comunicação.
- Efeitos da “Lei” de Amdahl.
- Gestão de cache e memória partilhada.

Processamento paralelo

Plataformas de multiprocessamento

- Objetivo: criar computadores potentes pela interligação de computadores mais simples:
 - programas devem aproveitar número variável de processadores
 - ênfase na utilização de processadores simples e eficientes
 - aumento da disponibilidade (robustez em face de falhas)
- Processador multinúcleo (*multi-core*): vários processadores (núcleos) num único circuito integrado.
- Vários processadores (multinúcleo ou não) podem ser reunidos num único computador: multiprocessador. [conceptualmente semelhante ao anterior]
- *Cluster*: computador paralelo composto por muitos processadores alojados em computadores independentes.

Processamento paralelo

- Paralelismo: execução simultânea de operações de processamento de dados possivelmente relacionadas (em geral).
- Como aproveitar uma plataforma de multiprocessamento?
 - Executar várias tarefas independentes em paralelo.
 - Decompôr uma tarefa em sub-tarefas que possam ser executadas em paralelo.
 - As sub-tarefas têm dependências entre si;
 - Geralmente, é necessária intervenção humana (programador) na decomposição.
- Vamos considerar a execução coordenada de sub-tarefas numa plataforma de multiprocessamento e dois grandes desafios (entre outros):
 - Como executar programas sequenciais eficientemente em hardware paralelo?
 - Como executar programas concorrentes eficientemente à medida que o número de processadores aumenta? (Escalabilidade)

Desafio 1: Maior rapidez de processamento

- Pretende-se obter um aumento de rapidez (*speed-up*) de 90 usando um computador com 100 processadores. Que percentagem dos cálculos pode ser sequencial?
- Lei de Amdahl: s : fator de aumento de rapidez aplicável
 p : percentagem de tempo que o melhoramento pode ser aplicado

$$S = \frac{1}{(1-p) + \frac{p}{s}}$$

- Sendo $S=90$, obtém-se $p \approx 0,999$
- Para obter um aumento de rapidez de 90 usando 100 processadores, apenas 0,1 % do tempo de execução pode ser sequencial.

Desafio 2: Escalabilidade

- Pretende-se somar duas matrizes de dimensão 10×10 e realizar 10 somas com variáveis simples (com dependências). Qual é o aumento de rapidez obtido usando um multiprocessador com 10 ou com 100 elementos?
(Seja t o tempo que demora fazer uma multiplicação uma adição).
- Tempo necessário num único processador sequencial: $(10 \times 10 + 10) \times t = 110 \times t$
- Tempo com multiprocessador 10: $(10 + 10) \times t = 20 \times t \Rightarrow S = 5,5$ (55,5 % do ideal)
- Tempo com multiprocessador 100: $(1 + 10) \times t = 11 \times t \Rightarrow S = 10$ (10 % do ideal)
- E se as matrizes forem 100×100 ? (nota: envolve 10000 adições)
 - Tempo necessário num único processador sequencial: $10010 \times t$
 - Tempo com multiprocessador 10: $1010 \times t \Rightarrow S = 9,9$ (99 % do ideal)
 - Tempo com multiprocessador 100: $110 \times t \Rightarrow S = 91$ (>90 % do ideal)

Escalabilidade forte e fraca

- *Escalabilidade de um algoritmo*

Escalabilidade forte: aumento de rapidez obtido
para problemas da mesma dimensão;

Escalabilidade fraca: aumento de desempenho obtido apenas aumentando também o problema de forma proporcional ao número de processadores.

- Seja M o espaço de memória e P o número de processadores:
 - escalabilidade forte: memória por processador M/P (decrece)
 - escalabilidade fraca: memória por processador M (mantém-se)
- O tipo de escalabilidade que é relevante depende do problema.

Desafio 3: Equilíbrio da distribuição de trabalho

- Até agora, assumiu-se que os cálculos podiam ser distribuídos de forma equilibrada por todos os processadores: para 100 processadores, cada um executa 1 % dos cálculos.
- O que sucede se um dos processadores tratar de 2 % dos cálculos (no caso de matrizes 100×100)?
- 2% de 10000 = 200 adições são feitas por um processador; os restantes 99 repartem 9800 adições.
- Tempo com multiprocessador com 100 processadores:

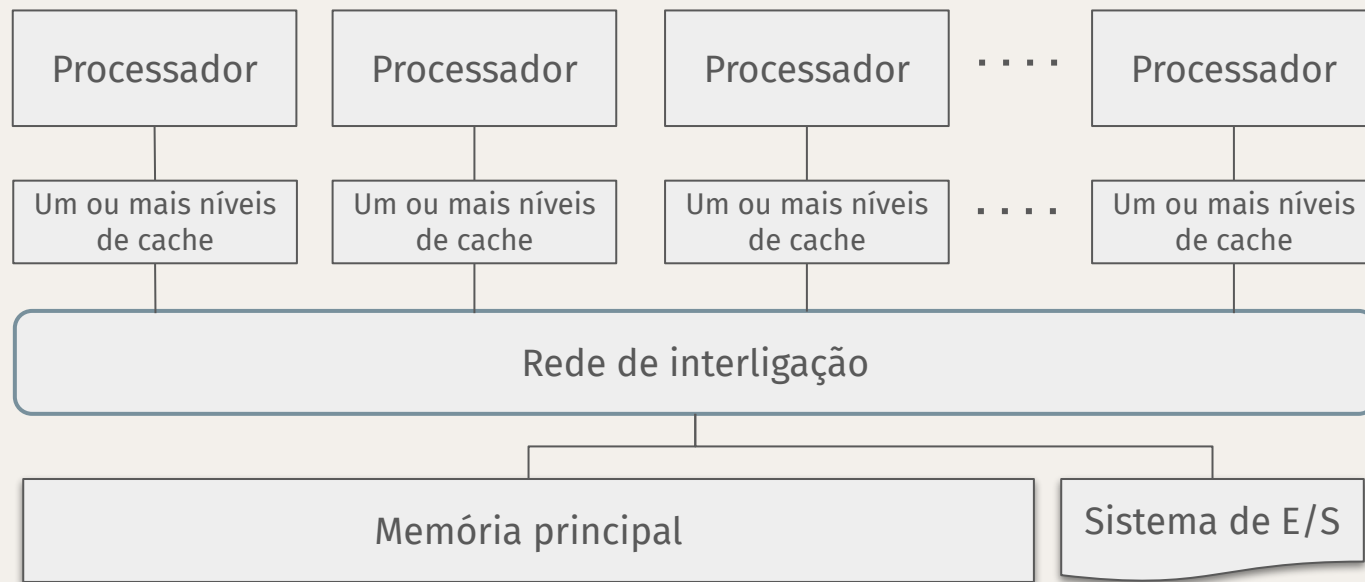
$$\max \left(\frac{9800 \times t}{99}, \frac{200 \times t}{1} \right) + 10 \times t = 210 \times t$$

$$\Rightarrow S = 48$$

Cerca de metade da situação equilibrada!

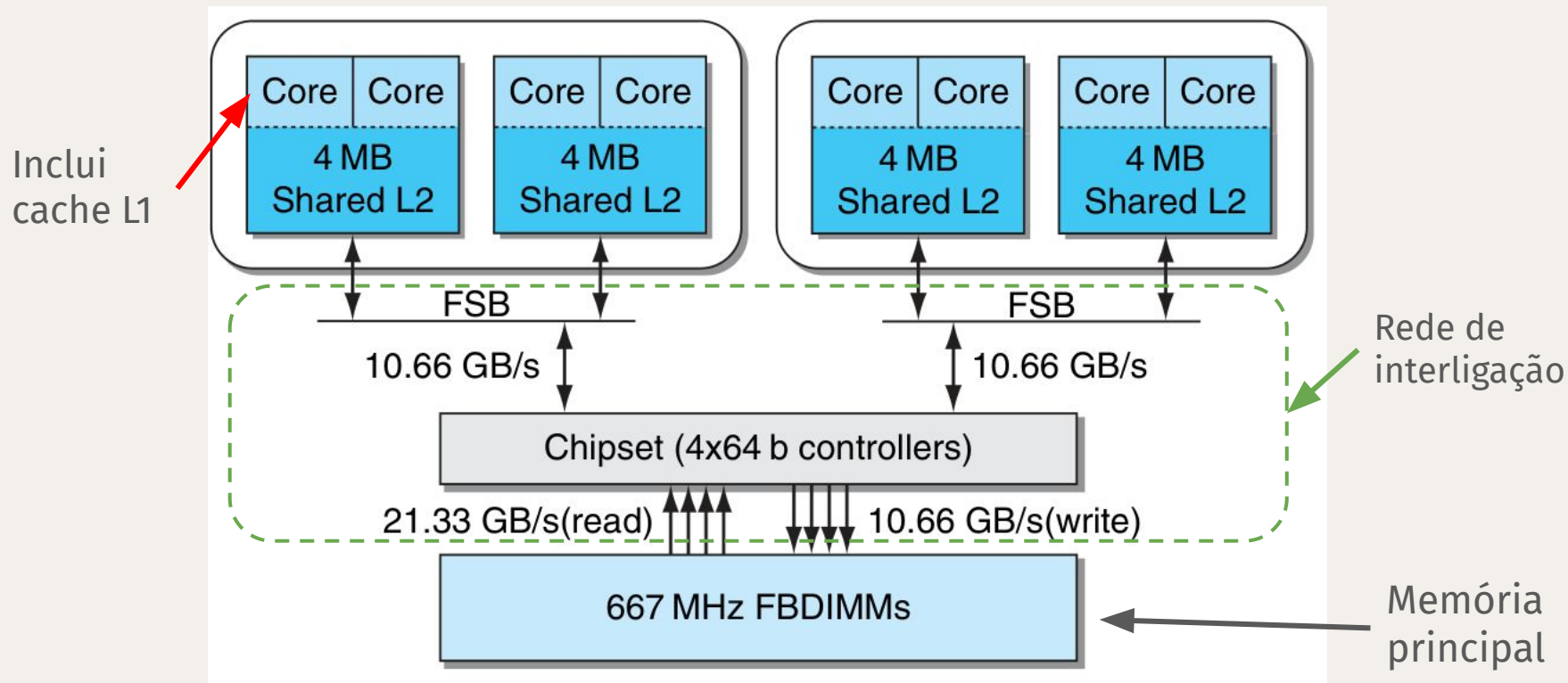
Multiprocessadores de Memória Partilhada

Multiprocessador simétrico de memória partilhada



- As memórias cache, desde que suficientemente grandes, permitem reduzir o número de acessos aos recursos comuns (rede de interligação e memória).
- Simétrico: significa que todos os processadores são idênticos.

Exemplo de uma arquitetura comercial: Xeon e5345



Caraterísticas

- Espaço físico de endereços único e igual para todos os processadores.
- Processadores comunicam através de variáveis partilhadas em memória: todos os processadores podem aceder a qualquer posição de memória via instruções *load/store*
- Necessidade de coordenação no acesso aos dados partilhados por código em execução em processadores diferentes: **sincronização** de acessos.
- Tempos de acesso a instruções/dados podem variar significativamente
 - Cache L1
 - Cache L2
 - Outros níveis de cache
 - Memória principal

Exemplo de um programa para memória partilhada (1)

- Tarefa: Somar 100000 números num multiprocessador com 100 processadores.
- Seja PN o identificador do processador ($0 \leq PN \leq 99$)
 - Durante a execução, o código a executar num processador tem acesso ao identificador
- Todos os processadores começam por executar o seguinte ciclo:
Cada um processa uma parte dos dados ($100000/100 = 1000$ números)

Primeira etapa:

```
sum[PN] = 0;  
for (i = 1000*PN; i < 1000*(PN+1); i=i+1)  
    sum[PN] = sum[PN] + A[i];
```

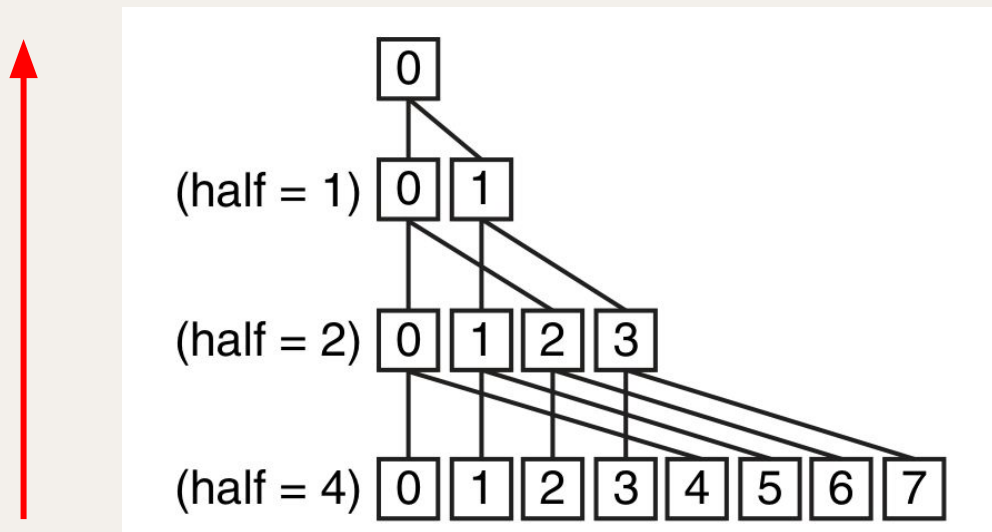
Programa para memória partilhada (2)

Segunda etapa (também em todos os processadores)

```
half = 100;  
repeat  
    synch(); // esperar pela conclusão da iteração em todos  
    if (half % 2  $\neq$  0 && PN == 0) // half é ímpar  
        sum[0] = sum[0] + sum[half-1]; // acumular em sum[0]  
    half = half / 2; // valor inteiro!  
    if (PN < half)  
        sum[PN] = sum[PN] + sum[PN + half];  
until (half == 1);
```


Programa para memória partilhada (3)

- A segunda etapa faz a adição das somas parciais (redução)
 - 1/2 dos processadores somam pares de números
 - 1/4 dos processadores somam as novas somas parciais
 - 1/8 dos processadores somam as novas somas parciais, etc.



O resultado final
fica em **sum[0]**

Coerência de memória cache

Memória partilhada comum

- A utilização de memórias cache grandes reduz os requisitos de largura de banda da memória principal e permite partilhar esta entre vários processadores via um barramento comum.
- A utilização de caches para acelerar o acesso a dados partilhados introduz o problema da **coerência de memória cache**.

Tempo	Evento	Cache A	Cache B	Mem X
0				1
1	CPU A lê X	1		1
2	CPU B lê X	1	1	1
3	CPU A escreve 0 em X	0	1	0

(Assumir cache *write-through*)

O valor existente na cache B não está correto.

Coerência

- Informalmente: Um sistema de memória é coerente se qualquer leitura de um item retornar o valor escrito mais recentemente.

Condições de coerência (valor produzido por uma leitura):

- Uma leitura do valor de X por P a seguir a uma escrita em X por P (sem escritas de outros processadores em X) retorna o valor escrito por P.
- Uma leitura do valor de X por P a seguir a uma escrita por outro processador retorna o novo valor (se operações de leitura e escrita estiverem suficientemente separadas no tempo).
- Escritas na mesma posição por dois processadores diferentes são “vistas” na mesma sequência por todos os processadores (serialização).

Como manter a coerência?

- Num multiprocessador, as memórias cache garantem *migração* e *replicação* de itens compartilhados.

Uma das estratégias mais usadas para manter a coerência é a Monitoração da rede de interligação (*snooping*) com invalidação na escrita.

- Monitoração: Cada cache que contém uma cópia de um bloco de memória também regista o seu estado “de partilha”; Todas as caches monitoram a rede de interligação.
- Invalidação na escrita: a cache que vai proceder à alteração assegura-se que detém acesso exclusivo ao item que vai ser modificado (causa a invalidação das cópias existentes em outras caches).

Protocolo de invalidação na escrita

- Para caches do tipo *write-back* com *allocate-on-write*
- Para proceder a uma invalidação, uma cache simplesmente coloca o endereço em questão na rede de interligação.

As outras caches invalidam cópias que eventualmente tenham.

- (+) Serialização é assegurada pela rede de interligação.
- (-) A escrita de um item partilhado necessita de acesso à rede.
- Onde ir buscar o valor de uma posição quando ocorre uma falta de cache?
 - Se existir outra cache com valor atualizado (dirty), esta envia o valor ao “requerente” e atualiza a memória principal.

Faltas de cache por partilha (faltas por coerência)

- As faltas que ocorrem devido à comunicação entre processadores são designadas por faltas por coerência.
A designação não é muito boa, já que não é a coerência que falha, mas é sim a necessidade de manter a coerência que causa as faltas.
- As faltas por coerência **verdadeiras** têm origem na partilha de dados entre processadores (via memória).
- As falhas por coerência **falsas** são devidas ao facto de as memória *cache* estarem organizadas em blocos:
 - Uma escrita de uma palavra num bloco causa a invalidação de todas as cópias do bloco; a leitura uma OUTRA palavra desse bloco por um dos processadores que possuía uma cópia do bloco original vai obrigar a uma actualização da cópia (que estava invalidada), apesar de o valor não ter, de facto, mudado.

Mais informação

- Secções 6.1-6.3 e 6.5 de David A. Patterson & John L. Hennessy
“Computer Organization and Design”, RISC-V edition, 2nd ed., Elsevier.